

A PYTHON IMPLEMENTATION OF THE EXTENDED EUCLIDEAN ALGORITHM

EDDIE DOOGAN

1. INTRODUCTION

Throughout this paper we will explore the Extended Euclidean Algorithm and Euclidean Domains such as the ring of polynomials over a field. These will be explored through the construction of a python implementation of the Extended Euclidean Algorithm which gives pretty output in the form of a .tex file.

It is presumed the reader is familiar with Rings and Integral Domains.

2. THEORY ON THE EXTENDED EUCLIDEAN ALGORITHM

Definition 2.1 (Euclidean Domain). Let R be an integral domain. A **Euclidean function** is a function $\phi : R \setminus \{0\} \mapsto \mathbb{N}$ such that for any $x, d \in R$, $d \neq 0$, there is some $q, r \in R$ such that $x = qd + r$ and either $r = 0$ or $\phi(r) < \phi(d)$. If a Euclidean function exists we say R is a Euclidean domain.[1]

Definition 2.2 (Euclidean Division). Let E a Euclidean Domain with Euclidean Function ϕ . For $x, d \in E$ with $d \neq 0$, we call $x = q \cdot d + r$ a Euclidean Division of x and d with respect to ϕ if $\phi(d) > \phi(r)$ or $r = 0$. If ϕ is clear from the context we can call it a Euclidean Division

Definition 2.3 (Divides). For a ring R and $x, y \in R$ we say ' x divides y ' often denoted ' $x|y$ ' if there exists some $r \in R$ such that $x \cdot r = y$

Definition 2.4 (Euclidean Greatest Common Divisor). Let E be a Euclidean Domain and $x, y \in E$. Then we say $g \in E$ is a **Greatest Common Divisor of x and y** if for any $r \in E$ that divides x and y , $r|g$. We let $gcds(x, y)$ denote the set of Greatest Common Divisors of x and y .

Lemma 2.5 (GCD is Unique up to Units). *Let E be a Euclidean Domain and $x, y \in E$. Then*

$$gcds(x, y) = \{g \cdot e : e \in E^\times\}$$

Where $E^\times$ is the set of units in E . [2]

Proof. Let $g_1 \in gcds(x, y)$, then $g|g_1$ and $g_1|g$, that is,

$$\begin{aligned} \exists a, b \in E \text{ s.t. } g \cdot a &= g_1 \text{ and } g_1 \cdot b = g \\ \implies (g_1 \cdot b) \cdot a &= g_1 \\ \implies a \cdot b &= 1 \\ \implies a, b &\in E^\times \end{aligned}$$

Thus we have $g_1 = g \cdot a \in \{g \cdot r : r \in E^\times\}$ and so, $\gcds(x, y) \subseteq \{g \cdot r : r \in E^\times\}$.
 For any $r \in \{g \cdot r : r \in E^\times\}$, we can express $r = g \cdot e$ with $e \in E^\times$.
 Suppose $g|k$ for some $k \in E$, then,

$$\begin{aligned} g \cdot p &= k && \text{For some } p \in E \\ g \cdot p \cdot e &= k \cdot e \\ r \cdot p &= (g \cdot e) \cdot p = k \cdot e && \text{By commutativity} \\ \implies r \cdot p \cdot e^{-1} &= k \end{aligned}$$

So we have that $r|k$. Since g is a Greatest Common Divisor, it divides any divisors of x and y , and so too does r , thus $r \in \gcds(x, y)$ and so $\{g \cdot r : r \in E^\times\} \subseteq \gcds(x, y)$. So overall, $\gcds(x, y) = \{g \cdot e : e \in E^\times\}$. $\square$

Algorithm 2.6 (Euclidean Algorithm). Let R be a Euclidean Domain with Euclidean function ϕ and $x, y \in R$. We describe the Euclidean Algorithm which finds a Greatest Common Divisor of x and y .

Algorithm 1 The Euclidean Algorithm

- 1: Relabel x and y if necessary so that $\phi(x) \geq \phi(y)$
 - 2: **while** $y \neq 0$ **do**
 - 3: Compute $q, r \in R$ such that $x = qy + r$ with $\phi(y) > \phi(r)$ or $r = 0$
 - 4: Set $x = y$ and $y = r$.
 - 5: **end while**
 - 6: **return** y which is now a GCD of the initial x and y
-

We can use the Euclidean Algorithm and multiply by units in the Ring[2.5] to find all the Greatest Common Divisors.

Notation 2.7. When referring to iterations of the Euclidean Algorithm we will denote iteration n by I_n (with I_0 being the first iteration) and values y_n and x_n to be those of x and y at the start of I_n . We will denote q_n and r_n to be the values of q and r calculated in iteration I_n and, when it is clear from context, N will refer to the total number of iterations conducted.

Lemma 2.8 (The Euclidean Algorithm Halts). *Let E be a Euclidean Domain and $x, y \in E$. Running the Euclidean Algorithm on x and y will halt.*

Proof. For any iteration I_n where the algorithm doesn't halt (i.e. $y_n \neq 0$), we calculate $\phi(y_{n+1}) = \phi(r_n) < \phi(y_n)$ so $\phi(y_{n+1}) \leq \phi(y_n) - 1$.
 By induction, if the algorithm hasn't halted by I_n , $\phi(y_n) \leq \phi(y_0) - n$.
 So the algorithm must halt before $I_{\phi(y_0)}$ otherwise, $\phi(y_n) \leq \phi(y_0) - \phi(y_0) = 0$ but ϕ has co-domain $\mathbb{N}$. $\square$

Lemma 2.9 (Euclidean Division relates to GCD). *Let E a Euclidean Domain and $x, d \in E$, $d \neq 0$. For a Euclidean Division $x = qd + r$, $\gcds(x, d) = \gcds(d, r)$.*

Proof. Let $CD(i, j)$ denote the common divisors of i and j for $i, j \in E$. For any $g_{xd} \in CD(x, d)$ there exist $a, b \in E$ s.t. $g_{xd} \cdot a = x$ and $g_{xd} \cdot b = d$. Then,

$$\begin{aligned} x &= qd + r \\ \implies r &= x - qd = g_{xd} \cdot a - q \cdot g_{xd} \cdot b \\ \implies r &= g_{xd}(a - qb) \end{aligned}$$

So $g_{xd}|r$ and thus, $CD(x, d) \subseteq CD(d, r)$. Similarly for any $g_{dr} \in CD(d, r)$ there exist $\alpha, \beta \in E$ s.t. $g_{dr} \cdot \alpha = d$ and $g_{dr} \cdot \beta = r$. Then,

$$x = qd + r = q \cdot g_{dr} \cdot \alpha + g_{dr} \cdot \beta = g_{dr}(q \cdot \alpha + \beta)$$

So $g_{dr}|x$ and thus, $CD(d, r) \subseteq CD(x, d)$. Overall we have that $CD(x, d) = CD(d, r)$ and thus $gcds(x, d) = gcds(d, r)$. $\square$

Theorem 2.10 (The Euclidean Algorithm is Correct). *For $x, d \in E$ the Euclidean Algorithm will return $g \in E$ such that $g \in gcds(x, d)$*

Proof. From 2.8 we know that a g will be returned. For each pair of consecutive iterations I_n, I_{n+1} we have

$$\begin{aligned} x_n &= q_n y_n + r_n \\ x_{n+1} &= q_{n+1} y_{n+1} + r_{n+1}. \end{aligned}$$

By 2.9, $gcds(x_n, y_n) = gcds(y_n, r_n)$ and, since $x_{n+1} = y_n$ and $y_{n+1} = r_n$, $gcds(x_n, y_n) = gcds(x_{n+1}, y_{n+1})$. Then by induction,

$$gcds(x_k, y_k) = gcds(x_j, y_j) \quad \forall k, j \in \mathbb{N} \text{ s.t. } k, j \leq N.$$

We have $gcds(x_0, y_0) = gcds(x_N, y_N) = gcds(y_N, r_N) = gcds(y_N, 0) = y_N$. $\square$

Corollary 2.11 (Existence of a Greatest Common Divisor). *For any a Euclidean Domain E and $x, y \in E$, $gcds(x, y) \neq \emptyset$.*

Proof. We can calculate a $g \in gcds(x, y)$ using the Euclidean Algorithm. $\square$

Example 2.12 (Nearing the Extended Algorithm). Consider the ring of Integers $(\mathbb{Z}, \cdot, +, 0, 1)$ with standard addition and multiplication ; and Euclidean Function $\phi(x) := |x|$ the absolute value. We can use the Euclidean Algorithm to find a GCD of 270 and 192,

$$\begin{aligned} 270 &= 1 \cdot 192 + 78 & I_0 \\ 192 &= 2 \cdot 78 + 36 & I_1 \\ 78 &= 2 \cdot 36 + 6 & I_2 \\ 36 &= 6 \cdot 6 + 0 & I_3 \end{aligned}$$

So we have a GCD of 6. Furthermore knowing that $\mathbb{Z}^\times = \{1, -1\}$ we can find $gcds(x, y) = \{6, -6\}$ by 2.5.

Using the system of equations we have created, we can back-put to find λ and μ such that

$$\lambda \cdot 270 + \mu \cdot 192 = 6,$$

$$\begin{aligned} 6 &= 78 - 2 \cdot 36 && \text{from } I_3 \\ &= 78 - 2 \cdot (192 - 2 \cdot 78) && \text{from } I_2 \\ &= 5 \cdot 78 - 2 \cdot 192 \\ &= 5 \cdot (270 - 1 \cdot 192) - 2 \cdot 192 && \text{from } I_3 \\ &= 5 \cdot 270 - 7 \cdot 192 \end{aligned}$$

So we have $\lambda = 5, \mu = -7$. This back-putting process is called the Extended Euclidean Algorithm.

Definition 2.13 (The Extended Euclidean Algorithm). Let E be a Euclidean Domain and $x, y \in E$. We aim to calculate $\lambda, \mu \in E$ s.t. $\lambda \cdot x + \mu \cdot y = g$, where g is the result from the Euclidean Algorithm on x and y .

Run the Euclidean Algorithm on x and y and keep track of the values of x_n, y_n, q_n, r_n for each iteration I_n :

$$\begin{aligned} x_0 &= q_0 \cdot y_0 + r_0 \\ x_1 &= q_1 \cdot y_1 + r_1 \\ &\vdots \\ x_{n-1} &= q_{n-1} \cdot y_{n-1} + r_{n-1} \\ x_n &= q_n \cdot y_n \end{aligned}$$

If the program only runs for one iteration we have that $g = y$ so trivially $0 \cdot x + 1 \cdot y = g$. If the program exists for more than one iteration, we use the facts that $x_k = y_{k-1}$ and $y_k = r_{k-1}$ for $n > k \in \mathbb{N}$ to create another system of equations

$$\begin{aligned} r_0 &= x_0 - q_0 \cdot y_0 \\ r_1 &= x_1 - q_1 \cdot y_1 \\ &\vdots \\ r_{n-1} &= x_{n-1} - q_{n-1} \cdot y_{n-1} \end{aligned}$$

Which using $y_n = x_{n+1}$ and $r_n = y_{n+1} \forall n \in \mathbb{N}, n < N$ becomes:

$$\begin{aligned} y_1 &= x_0 - q_0 \cdot y_0 \\ y_2 &= y_0 - q_1 \cdot y_1 \\ &\vdots \\ y_n &= y_{n-2} - q_{n-1} \cdot y_{n-1} \end{aligned}$$

We will call this system the **Equations** of x and y .

We can use these Euclidean Equations and back-subbing to calculate our desired λ and μ , such as has been done in [2.12](#). This back-subbing using the equations of x and y is called the **Extended Euclidean Algorithm**.

Algorithm 2.14 (Recursive Extended Euclidean Algorithm). Let E a Euclidean Domain and $x, y \in E$. the following is a recursive implementation of the Extended Euclidean Algorithm on x and y ,

Algorithm 2 Recursive Extended Euclidean Algorithm

```

1: if  $\phi(y) > \phi(x)$  then
2:   Swap  $x$  and  $y$ 
3: end if
4:
5: procedure RECURSION( $n$ )
6:   if  $n == N - 2$  then
7:     // Base Case
8:     return  $(1, -q_n)$ 
9:   end if
10:  Let  $\lambda, \mu = \text{RECURSION}(n+1)$ 
11:  return  $(\mu, (\lambda - \mu \cdot q_{n-1}))$ 
12: end procedure
13:
14: Run the Euclidean Algorithm with  $x, y \in E$ , and record  $q_n$  for each iteration,  $I_n$ .
15: Let  $N$  be the number of iterations.
16: if  $N == 1$  then
17:   Set  $\lambda = 0, \mu = 1$ 
18: end if
19: if  $N > 1$  then
20:    $\lambda, \mu = \text{RECURSION}(0)$ 
21: end if
22: if  $x$  and  $y$  were swapped then
23:   Swap  $\lambda$  and  $\mu$ 
24: end if
25: return  $(\lambda, \mu)$ 

```

Remark 2.15. The recursive algorithm can be easily converted to a looping algorithm, by starting at the end of the q s and successively calculating the new λ, μ each time.

Lemma 2.16. *Suppose we have two equations,*

$$\begin{aligned} c &= a - qb \\ d &= \alpha \cdot b + \beta \cdot c \end{aligned}$$

then

$$\begin{aligned} d &= \alpha \cdot b + \beta \cdot (a - qb) \\ \implies d &= (\alpha - \beta \cdot q)b + \beta \cdot a \end{aligned}$$

Lemma 2.17 (Recursion Correctness). *Let E a Euclidean Domain and $x, y \in E$, $\phi(x) > \phi(y)$ such that the Euclidean Algorithm on x and y iterates more than once. Then, for any n , a valid iteration number, $\text{RECURSION}(n)$ will return λ, μ such that $\lambda \cdot x_n + \mu \cdot y_n = g$, where g is the output GCD from the Euclidean Algorithm.*

Proof. We prove by induction using 2.16.

Base Case (RECURSION(N-2)):

The last Equation (2.13) of x and y states $r_{n-1} = x_{n-1} - q_{n-1} \cdot y$ which together with $g = y_{N-1} = r_{N-2}$ gives $g = x_{n-1} - q_{n-1}$ so $\lambda = 1, \mu = -q_{N-1}$ are valid, which is the output of RECURSION(N-2).

Inductive Step :

Assume the statement is true for RECURSION(n) with $N - 2 > n \in \mathbb{N}$.

Then, with the Equation for x_{n-1} and y_{n-1} , we have

$$\begin{aligned} y_n &= x_{n-1} - q_{n-1} \cdot y_{n-1} \\ g &= \lambda \cdot x_n + \mu \cdot y_n \end{aligned}$$

Which together with the fact $x_k = y_{k-1} \forall k \in \mathbb{N}, k < N - 1$ gives,

$$\begin{aligned} g &= \lambda \cdot y_{n-1} + \mu \cdot (x_{n-1} - q_{n-1} \cdot y_{n-1}) \\ \implies g &= \mu \cdot x_{n-1} + (\lambda - \mu \cdot q_{n-1}) y_{n-1} \end{aligned}$$

Which are precisely the values returned for RECURSION(n-1). □

Theorem 2.18. *The algorithm 2.14 is correct.*

Proof. Let E a Euclidean Domain and $x, y \in E$. Since if $\phi(x) < \phi(y)$ the program swaps them at the start and swaps λ and μ at the end, we can assume without loss of generality $\phi(x) \geq \phi(y)$.

If the Euclidean Algorithm is one iteration then $g = y$ and so $\lambda = 0, \mu = 1$ are valid multipliers which are set on line 17.

Otherwise, 2.17 provides the result that we produce a λ, μ that work for $x_0 = x$ and $y_0 = y$ □

Remark 2.19. The only values used in the Extended Euclidean Algorithm are the q s, indeed rather than record the whole system of equations generated by the Euclidean Algorithm, we can record only the q s and, without even knowing the values of x and y we are searching for coefficients for, we can find them.

3. PYTHON FIELD AND EUCLIDEAN DOMAIN IMLEMENTATIONS

The Python Implementation to which this paper will regularly refer can be found in A or hosted at <https://github.com/Eddie-D/MTH3012-Extended-Euclidean-Algorithm>, the latter of which is the recommended means of access.

Remark 3.1 (On Choosing Python). The intention for the program is to create an implementation of the Extended Euclidean Algorithm that can run on a variety of rings and can provide a clean, readable output which will be done through use of L^AT_EX.

There are some programming paradigms that would make the program easier:

- (1) **Object Orientated** - The Extended Euclidean Algorithm works on any Euclidean Domain. It would be intuitive to program it over an arbitrary Euclidean Domain using an interface and then implement this interface to define our rings.

- (2) **Permits Operator Overloading** - A ring can be thought of as having a $+$ and $*$ operation. Overloading the operators for each ring allows the code to be written intuitively using $*$ in place of $\cdot$ and $+$ as $+$ (and in many languages obeying expected precedence).

Two languages stand out as fitting these criteria, C++ and Python. C++ is a better choice than Python as Python's Duck Typing loses it many of the compile time benefits of interfaces. Nonetheless, the program has been made in python as my familiarity with it outweighs the benefits C++ offers.

Data Structure 3.2 (The Rings Requirements). Each Euclidean Domain, E , with Euclidean Function ϕ , that is not a field will have the following:

- (1) var unit the ring's multiplicative unit
- (2) var zero the ring's additive unit
- (3) def console_element() returns an element of E created from console input
- (4) def euclideanFunction($x \in E$) returns $\phi(x)$
- (5) def euclideanDivision($x \in E, y \in E$) returns q, r from the euclidean division of x and y (2.1).

With fields having the same except euclideanFunction and euclideanDivision as they are trivial.

Data Structure 3.3 (The Ring Elements). Each element of a ring must have the following Python dunder methods implemented:

- (1) `__add__(x,y)` performs ring addition.
- (2) `__sub__(x,y)` performs ring subtraction.
- (3) `__mul__(x,y)` performs ring multiplication.
- (4) `__neg__(x)` returns the additive inverse.
- (5) `__str__(x)` returns x in a latex-math-mode string.
- (6) `__eq__(x,y)` checks if two elements are the same.

and Fields will also implement:

- (7) `__truediv__(x,y)` returns z s.t. $z \cdot y = x$

Example 3.4 (Field Data Structure). The code for the field $\mathbb{Z}/p\mathbb{Z}$ is,

```

1 class ZpRing :
2     def __init__(self, p=None) :
3         if not p :
4             print("Please choose p for Z/p")
5             p = tillValid(lambda: int(input()), "Please choose an integer
        mod p")
6         self.p = p
7         self.unit = Zp(self.p, 1)
8         self.zero = Zp(self.p, 0)
9
10    def console_element (self) :
11        return Zp(self.p, tillValid(lambda: int(input()), f"Please choose
        an integer mod {self.p}"))
12

```

Which is not especially interesting, but shows the requirements laid out in 3.2 being fulfilled. The *tillValid* method repeats a function until it doesn't throw an error, showing the text in between repeats. It's code can be seen in appendix A.

The code for the elements of $\mathbb{Z}/p\mathbb{Z}$ is,

```

1 class Zp:
2     def __init__ (self, p, value) :
3         self.p = p
4         self.value = int(value) % p
5
6     def check_arith_match(self, other) :
7         if self.p != other.p : raise Exception("Modular arithmetic
8             mismatch")
9
10    def __add__ (self, other) :
11        self.check_arith_match(other)
12        return Zp(self.p, self.value + other.value)
13
14    def __sub__ (self, other) :
15        self.check_arith_match(other)
16        return Zp(self.p, self.value - other.value)
17
18    def __mul__ (self, other) :
19        self.check_arith_match(other)
20        return Zp(self.p, self.value * other.value)
21
22    def __neg__ (self):
23        return Zp(self.p, self.p - self.value)
24
25    def __str__ (self) :
26        return f"[{self.value}]_{{{self.p}}}"
27
28    def __truediv__(self, other) :
29        # Calculate the inverse by fermats little theorem
30        self.check_arith_match(other)
31        for _ in range(self.p - 2) :
32            self *= other
33        return self
34
35    def __eq__(self, other):
36        # Don't check arithmatch as we instead use n to check if values
37        are equal
38        return self.value == other.value and self.p == other.p

```

At initialization, the element's value is squashed to be between 0 and $p - 1$, using remainder arithmetic to unify representations, allowing comparisons to be made by comparing the value.

An interesting feature is the use of Fermat's Little Theorem[1] to calculate the inverse in *__truediv__*.

Example 3.5 (Euclidean Domain Data Structure). The implementation of the Euclidean Domain that is the Integers, uses the already implemented dunder methods of python integers. We have code for the integer ring,

```

1 # IntegerRing is static as it doesn't need paramaters at initialisation
2 class IntegerRing :
3     zero = 0
4     unit = 1
5     @staticmethod
6     def console_element() :
7         return tillValid(lambda: int(input()), f"Please choose an integer"
8     )
9
10    @staticmethod
11    def euclideanFunction(x) :
12        return abs(x)
13
14    @staticmethod
15    def euclideanDivision(x,y) :
16        return (x // y, x % y)

```

Remark 3.6. The implementation shows why the *euclideanFunction* is declared on the Ring instead of the elements. Since the *euclideanFunction* only takes in one element, intuition would put it in the element class, where it could be called without needing any arguments. However, leaving the function in the ring allows us to use pre-built python classes from libraries to quickly create rings that implement the required dunder methods, such as *int* from the standard library.

4. PYTHON IMPLEMENTATION OF THE EXTENDED EUCLIDEAN ALGORITHM

Having defined our requirements for Euclidean Domains in 3.2, we can construct a python implementation of both the Euclidean and Extended Euclidean Algorithms.

Algorithm 4.1 (Python Euclidean Algorithm). The code for the python implementation of the Euclidean Algorithm is,

```

1 def euclidean_algorithm(ring, e1, e2, writer=None) :
2     # Assign divisor and dividend where 0 keeps track of the originals
3     dividend0, divisor0 = (e1, e2) if ring.euclideanFunction(e1) >= ring.
4     euclideanFunction(e2) else (e2, e1)
5     dividend, divisor = dividend0, divisor0
6
7     if writer :
8         writer.write(f"\textbf{{We use the Euclidean Algorithm to find
9         the gcd of ${divisor0}$ and ${dividend0}$}} \\\n")
10        writer.write("\begin{align*} \n")
11        results = []
12        remainder = None
13
14        while (remainder is None or remainder != ring.zero) :
15            quotient, remainder = ring.euclideanDivision(dividend, divisor)

```

```

14         results.append([dividend, quotient, divisor])
15
16         if writer :
17             writer.write(f"{dividend} &= ({quotient})(divisor) + ({
remainder}) \\\n")
18
19         dividend = divisor
20         divisor = remainder
21
22     gcd = dividend
23     if writer :
24         writer.write("\\end{align*} \n")
25         writer.write(f"\\textbf{{Overall the gcd is ${gcd}$}} \\\n")
26
27     return { "gcd":gcd, "results":results }
28

```

where the input arguments are *ring*, the Euclidean Domain; *e1* and *e2* the elements we want to find the *gcd* of; and *writer* an optional argument that when provided will have the process written to.

Remark 4.2. The algorithm is clearly the same as 2.6 having two notable features absent from 2.6.

The algorithm has a 'writer' scattered throughout. When provided, the writer is used to output the steps of the algorithm in a well-formatted manner using the *writer.write* def.

The algorithm returns two values, the expected *gcd*, and a 'results' value which stores the dividend, quotient and divisor for each iteration. A record of the quotients needs to be kept for the Extended Euclidean Algorithm (see 2.19), the others are kept to permit a prettier output from the Extended Euclidean Algorithm.

Algorithm 4.3 (Python Extended Euclidean Algorithm). The code for the python implementation of the Extended Euclidean Algorithm is,

```

1 def extended_euclidean_algorithm(ring, e1, e2, writer=None) :
2     euclid = euclidean_algorithm(ring, e1, e2, writer)
3     gcd, results = euclid["gcd"], euclid["results"]
4     dividend0, divisor0 = (e1, e2) if ring.euclideanFunction(e1) >= ring.
euclideanFunction(e2) else (e2, e1)
5
6     # Remove result with zero remainder
7     results.pop()
8
9     # Design is discussed in the report
10    def extended_euclidean(n) :
11        dividend, quotient, divisor = results[n]
12        if n == len(results) - 1 :
13            lamb, mu = ring.unit, -quotient
14        else :
15            # Swap and calculate new mu
16            mu, lamb = extended_euclidean(n+1)
17            mu = mu - (quotient * lamb)

```

```

18
19         if writer :
20             writer.write(f"{gcd} &= ({lamb})({dividend}) + ({mu})({divisor}
21             )) \\ \\ \\n")
22         return (lamb, mu)
23
24     if(len(results) != 0) :
25         if writer :
26             writer.write(f"\\textbf{{We use the Extended Euclidean
27             algorithm to find coefficients $\\lambda$, $\\mu$ s.t. $\\lambda ({
28             dividend0}) + \\mu ({divisor0}) = {gcd}$}} \\ \\ \\n")
29             writer.write("\\begin{align*} \\n")
30
31             lamb, mu = extended_euclidean(0)
32
33         if writer:
34             writer.write("\\end{align*} \\n")
35             writer.write(f"\\textbf{{Overall we have: $\\lambda = ({lamb})$
36             and $\\mu = ({mu})$}} \\ \\ \\n")
37         else :
38             # In trivial case, mu is the gcd
39             lamb, mu = ring.zero, ring.unit
40
41         if writer :
42             writer.write(f"\\textbf{{We have trivially that $({lamb})({
43             dividend0}) + ({mu})({divisor0}) = {gcd}$}} \\ \\ \\n")
44
45

```

where the input arguments are the same as for the Euclidean Algorithm. The implementation is clearly the same as 2.14 with the addition of the writer and absence of return values. Though the swap-back of μ and λ in 2.14 could easily be done, and the values returned, nowhere in our code do we need to use the values calculated by the algorithm, they're only used when written.

Algorithm 4.4 (Driver). At this point we can create a small driver program to get results from our code,

```

1 from rings import getEuclideanDomain
2 from extended_euclidean import extended_euclidean_algorithm
3
4 # Setup ring-specific variables
5 ring = getEuclideanDomain()
6
7 print("Enter the two elements to run the Extended Euclidean Algorithm on")
8 print("Enter the first element:")
9 e1 = ring.console_element()
10
11 print("Enter the second element:")
12 e2 = ring.console_element()
13
14 path = "./ExtendedEuclideanOutput.tex"

```

```

15
16 # Open a file writer
17 writer = open(path, "w")
18 writer.write("\\textbf{Extended Euclidean Algorithm,} \\\\n")
19 extended_euclidean_algorithm(ring, e1, e2, writer=writer)
20 print("Extended Euclidean Algorithm Completed successfully, see
      ExtendedEuclideanOutput.tex")
21 writer.close()
22

```

Example 4.5 (Extended Euclidean Algorithm on Integers). We run the driver code with the ring $\mathbb{Z}$ and elements, $270, 192 \in \mathbb{Z}$ which have a $\gcd = 6$ which can be achieved by $5 \cdot 270 - 7 \cdot 192$ (see 2.12). The terminal for doing this is,

```

1 "Please choose a Ring to operate in (Z (Integers), F[X] (Polynomials over
   a Field))"
2 Z
3 "Enter the two elements to run the Extended Euclidean Algorithm on"
4 "Enter the first element:"
5 270
6 "Enter the second element:"
7 192
8 "Extended Euclidean Algorithm Completed successfully, see
   ExtendedEuclideanOutput.md"
9

```

and the resulting latex is,

ExtendedEuclideanOutput.tex

Extended Euclidean Algorithm,

We use the Euclidean Algorithm to find the gcd of 192 and 270

$$270 = (1)(192) + (78)$$

$$192 = (2)(78) + (36)$$

$$78 = (2)(36) + (6)$$

$$36 = (6)(6) + (0)$$

Overall the gcd is 6

**We use the Extended Euclidean algorithm to find coefficients λ, μ s.t.
 $\lambda(270) + \mu(192) = 6$**

$$6 = (1)(78) + (-2)(36)$$

$$6 = (-2)(192) + (5)(78)$$

$$6 = (5)(270) + (-7)(192)$$

Overall we have: $\lambda = (5)$ and $\mu = (-7)$

5. PYTHON RINGS IN RINGS IMPLEMENTATIONS

For this section, it is presumed the reader is familiar with:

- Polynomial Rings
- Polynomial Long Division
- Polynomial Rings over a Field form a Euclidean Domain with Euclidean Function $\deg$ where q, r can be calculated using Polynomial Long Division.
- Ring of Fractions over and Integral Domain

Each of which can be found in the lecture notes [1]

Theorem 5.1. *Let E a Euclidean domain with euclidean function ϕ , then $\mu : E \rightarrow \mathbb{N}$*

$$\mu(x) := \begin{cases} 1 & \text{if } x = 0 \\ \phi(x) + 1 & \text{otherwise} \end{cases}$$

is a Euclidean Function on E that can use the same division as ϕ .

Proof. From definition, for any $x, d \in E$, $d \neq 0$ we can find $q, r \in E$ such that $x = qd + r$. If $r = 0$ we are done, otherwise $\phi(d) > \phi(r)$, then $\mu(y) = \phi(d) + 1 > \phi(r) + 1 = \mu(r)$ since $d, r \neq 0$. $\square$

Algorithm 5.2 (Python Polynomial Elements Implementation). An advantage of having a 'blueprint' in form of our Ring Requirements (3.2) is that we can create rings over other rings, such as the ring of polynomials over a ring. The code for the elements of the Polynomial Ring is,

```

1 class Polynomial :
2     # We store polynomials as a + bx + cx^2 + ... + dx^n = [a b c ... d]
3     def __init__(self, ring, values, indeterminate="X") :
4         self.indeterminate = indeterminate
5         # The underlying ring
6         self.ring = ring
7         # Remove leading zeroes
8         while len(values) > 0 and values[-1] == ring.zero:
9             values.pop()
10        self.values = values
11
12    def length(self) :
13        return len(self.values)
14
15    def strTerm(self, coeff, pow) :
16        if coeff == self.ring.zero:
17            return ""
18        elif pow >= 1:
19            strCo = str(coeff) if coeff != self.ring.unit else ""
20            strX = f"{self.indeterminate}^{{{pow}}}" if pow > 1 else f"{
self.indeterminate}"
21            return strCo + strX
22        else :
23            return str(coeff)
24
25    def __str__(self) :
26        if self.values == [] :
27            # Return constant zero for zero polynomial
28            return str(self.ring.zero)
29
30        # Create string representation of polynomial
31        # TODO Check this is a smart implementation
32        rVals = [self.values[-i] for i in range(1, self.length() + 1)]
33        strP = [self.strTerm(rVals[i], len(rVals)-i-1) for i in range(0,
len(rVals))]
34        return " + ".join(filter(lambda s: s != "", strP))
35
36    # We use coeffwise to simplify the other methods
37    def coeffwise(self, other, op) :
38        length = max(self.length(), other.length())
39
40        def withLeadingZeroes(l, length) :
41            # Add leading zeroes till the list goes up to the desired
length
42            return l + [self.ring.zero] * (length - len(l))
43
44        p1, p2 = withLeadingZeroes(self.values, length), withLeadingZeroes
(other.values, length)
45        return Polynomial(self.ring, [op(p1[i], p2[i]) for i in range(
length)], self.indeterminate)

```

```

46
47     def __add__ (self, other) :
48         return self.coeffwise(other, operator.add)
49
50     def __sub__ (self, other) :
51         return self.coeffwise(other, operator.sub)
52
53     def __mul__ (self, other) :
54         total = Polynomial(self.ring, [self.ring.zero], self.indeterminate
55         )
56         for i, x in enumerate(self.values) :
57             total = total + Polynomial(self.ring, [self.ring.zero] * i + [
58                 x*y for y in other.values], self.indeterminate)
59         return total
60
61     def __eq__ (self, other) :
62         # Efficient for polynomials of different lengths
63         if self.length() != other.length() :
64             return False
65         # Iterate over self and other simultaneously
66         for x, y in zip(self.values, other.values) :
67             if x != y :
68                 return False
69         return True
70
71     def __neg__ (self) :
72         return Polynomial(self.ring, [-x for x in self.values], self.
73         indeterminate)
74
75     def highestCoeff(self) :
76         return self.values[-1] if self.length() > 0 else self.ring.zero
77
78

```

Remark 5.3 (Coeffwise). The code creates a def, `coeffwise`, that takes an operator and two elements and applies the operator on the two elements 'coefficient-wise'. Rather than writing out individual implementations for `__add__` and `__sub__` we can use the mathematical language that they are done 'coefficient-wise' to reduce their code, making it more readable.

Remark 5.4 (String Conversion). The conversion to string calls a helper def, `strTerm`. `strTerm` gives a single term as a string, excluding it if it has a coefficient of zero, not writing the coefficient if it is one, and combining the coefficient with its X^n term, formatted well. This allows the `__str__` method to return a well-formatted version by placing "+" between the terms. The exception is the zero polynomial, for which all terms will return blank, this is caught by an edge-case in the `__str__` def.

Remark 5.5. The code uses a euclidean function of $length(x) = 1 + deg(x)$ which is valid by $deg(x)$ being a euclidean function[1] and 5.1

Algorithm 5.6 (Python Polynomial Ring Implementation). Though our code for an element of a Polynomial Ring allows polynomials to be created over any ring, not just a field, the code for the Polynomial Ring itself requires the underlying ring be a field. This is so that we can

define the Euclidean Division using polynomial long division, which requires an underlying field. The code for the ring is,

```

1 # Ring of polynomials, Character is the letter used for the unknown and
  ring is the underlying ring
2 class PolyRing :
3     def __init__(self, ring=None, character=None) :
4         if not character :
5             print("Enter a letter to represent the indeterminate (
  typically X)")
6             def getChar() :
7                 c = input()
8                 if len(c) != 1 or not c.isalpha() :
9                     raise Exception("Not a character")
10                return c
11            self.indeterminate = tillValid(getChar, "Enter an alphabetical
  character")
12
13            ring = getField()
14
15            self.ring = ring
16            self.zero = Polynomial(ring, [ring.zero], self.indeterminate)
17            self.unit = Polynomial(ring, [ring.unit], self.indeterminate)
18
19            def console_element (self) :
20                print("Enter a degree for the polynomial")
21                degree = tillValid(lambda: int(input()), "Enter an integer degree"
  )
22
23                values = []
24                for i in range(degree,-1,-1) :
25                    print(f"Enter the coefficient for {self.indeterminate}^{i} by
  entering a ring value:")
26                    values.insert(0, self.ring.console_element())
27                return Polynomial(self.ring, values, self.indeterminate)
28
29            # The euclidean division over polynomials is polynomial long division
30            def euclideanDivision(self, dividend, divisor) :
31                remainder = dividend
32                quotient = self.zero
33
34                while(divisor.length() <= remainder.length()) :
35                    # Create the monomial multiplier
36                    mult = Polynomial(self.ring, [self.ring.zero] * (remainder.
  length() - divisor.length()) + [remainder.highestCoeff() / divisor.
  highestCoeff()], self.indeterminate)
37                    remainder = remainder - (divisor * mult)
38                    quotient = quotient + mult
39
40                return (quotient, remainder)

```

```

41
42     # Not the traditional euclidean function in F[X] which is deg(f), but
    is a valid one nonetheless
43     @staticmethod
44     def euclideanFunction(p) :
45         return p.length()
46

```

Example 5.7 (Extended Euclidean Algorithm on Polynomials). We can use our driver file to run the Extended Euclidean Algorithm on $\mathbb{Z}/p\mathbb{Z}[X]$. We run the algorithm on $X^3 + X^2 + [3]_5X$ and $X^3 + [3]_5X^2 + [4]_5X + [2]_5$. The terminal execution is,

```

1  "Please choose a Ring to operate in (Z (Integers), F[X] (Polynomials over
    a Field))"
2  f
3  "Enter a letter to represent the indeterminate (typically X)"
4  X
5  "Please choose a field to operate over (Frac[E] (Fractions over a
    Euclidean Domain), Z/p (Integers mod P))"
6  zp
7  "Please choose p for Z/p"
8  5
9  "Enter the two elements to run the Extended Euclidean Algorithm on"
10 "Enter the first element:"
11 "Enter a degree for the polynomial"
12 3
13 "Enter the coefficient for X^3 by entering a ring value:"
14 1
15 "Enter the coefficient for X^2 by entering a ring value:"
16 3
17 "Enter the coefficient for X^1 by entering a ring value:"
18 4
19 "Enter the coefficient for X^0 by entering a ring value:"
20 2
21 "Enter the second element:"
22 "Enter a degree for the polynomial"
23 3
24 "Enter the coefficient for X^3 by entering a ring value:"
25 1
26 "Enter the coefficient for X^2 by entering a ring value:"
27 1
28 "Enter the coefficient for X^1 by entering a ring value:"
29 3
30 "Enter the coefficient for X^0 by entering a ring value:"
31 0
32 "Extended Euclidean Algorithm Completed successfully, see
    ExtendedEuclideanOutput.md"

```

Which gives the output file,

ExtendedEuclideanOutput.tex

Extended Euclidean Algorithm,

We use the Euclidean Algorithm to find the gcd of $X^3 + X^2 + [3]_5X$ and $X^3 + [3]_5X^2 + [4]_5X + [2]_5$

$$\begin{aligned} X^3 + [3]_5X^2 + [4]_5X + [2]_5 &= ([1]_5)(X^3 + X^2 + [3]_5X) + ([2]_5X^2 + X + [2]_5) \\ X^3 + X^2 + [3]_5X &= ([3]_5X + [4]_5)([2]_5X^2 + X + [2]_5) + ([3]_5X + [2]_5) \\ [2]_5X^2 + X + [2]_5 &= ([4]_5X + [1]_5)([3]_5X + [2]_5) + ([0]_5) \end{aligned}$$

Overall the gcd is $[3]_5X + [2]_5$

We use the Extended Euclidean algorithm to find coefficients λ, μ s.t.
 $\lambda(X^3 + [3]_5X^2 + [4]_5X + [2]_5) + \mu(X^3 + X^2 + [3]_5X) = [3]_5X + [2]_5$

$$\begin{aligned} [3]_5X + [2]_5 &= ([1]_5)(X^3 + X^2 + [3]_5X) + ([2]_5X + [1]_5)([2]_5X^2 + X + [2]_5) \\ [3]_5X + [2]_5 &= ([2]_5X + [1]_5)(X^3 + [3]_5X^2 + [4]_5X + [2]_5) + ([3]_5X)(X^3 + X^2 + [3]_5X) \end{aligned}$$

Overall we have: $\lambda = ([2]_5X + [1]_5)$ and $\mu = ([3]_5X)$

Remark 5.8. The console_element for polynomials works fine, however it cause a much bigger input sequence than that in Example 4.5. This is necessary due to the amount of coefficients we have to input, but can make inputting error-prone. The terseness will become even more evident after we implement the ring of Fractions.

Definition 5.9 (Lowest Form of Fractions). Let E a Euclidean Domain and $\mathbb{F} = \text{Frac}(E)$. We say $\frac{e_1}{e_2} \in \mathbb{F}$ is in its **lowest form** if the Greatest Common Divisors of e_1, e_2 are units.

Algorithm 5.10 (Algorithm for Finding Lowest Form of a Fraction). Let E a Euclidean Domain and $\mathbb{F} = \text{Frac}(E)$. For any $\frac{e_1}{e_2} \in \mathbb{F}$ we can use the following algorithm to calculate one of its lowest forms.

Algorithm 3 Lowest form of Euclidean Fraction

- 1: Let g be a gcd of e_1, e_2 calculated using the euclidean algorithm.
 - 2: Find q_1, q_2 such that, $g \cdot q_1 = e_1$ and $g \cdot q_2 = e_2$.
 - 3: return $\frac{q_1}{q_2}$
-

We can find all other lowest forms by multiplying the numerator and denominator by the units in E .

Proof. Let $g \in \text{gcds}(e_1, e_2)$ calculated using the euclidean algorithm. Then there exists q_1, q_2 such that $g \cdot q_1 = e_1$ and $g \cdot q_2 = e_2$. Examine $x \in \text{gcds}(q_1, q_2)$, there exists x_1, x_2 such that $x \cdot x_1 = q_1$ and $x \cdot x_2 = q_2$. Then $g \cdot x \cdot x_1 = e_1$ and $g \cdot x \cdot x_2 = e_2$ so $g \cdot x | e_1$ and $g \cdot x | e_2$, thus $g \cdot x \in \text{gcds}(e_1, e_2)$ so by 2.5, x is a unit. Then also by 2.5, $\text{gcds}(q_1, q_2) = \mathbb{F}^\times$. $\square$

Corollary 5.11 (Existence of Lowest Form). *Let E a Euclidean Domain and $\mathbb{F} = \text{Frac}(E)$. Then any $\frac{e_1}{e_2} \in \mathbb{F}$, has a lowest form.*

Proof. Use algorithm 5.10 to find a lowest form. $\square$

Algorithm 5.12 (Python Implementation of Ring of Fractions). We have the following implementation for the ring of fractions over a euclidean domain in python,

```

1 class FractionRing :
2     def __init__(self, ring=None) :
3         if not ring :
4             ring = getEuclideanDomain()
5             self.unit = Frac(ring, ring.unit, ring.unit)
6             self.zero = Frac(ring, ring.zero, ring.unit)
7             self.ring = ring
8
9     def console_element (self) :
10        if self.ring == IntegerRing :
11            return tillValid(lambda: Frac(self.ring, *[int(x) for x in
12                input().split("/")]), "Enter a fraction in the form X/Y for integers X
13                and Y")
14
15        else :
16            print("Enter the numerator of the Fraction")
17            num = self.ring.console_element()
18            print("Enter the denominator of the Fraction")
19            den = self.ring.console_element()
20            return Frac(self.ring, num, den)

```

Remark 5.13. The console_element def has two cases, when the underlying ring is the integers and otherwise. In the former, elements are entered as 'x/y' whereas in the latter they are entered in two lines as a numerator and denominator. This allows some reduction in the length of the console input, making it more intuitive. The numerator and denominator case is necessary for other rings, such as polynomials, where a '/' would serve to make the input less intuitive.

Algorithm 5.14 (Python Implementation of Fraction Elements). We have the following implementation for the elements of a ring of fractions over a euclidean domain in python,

```

1 class Frac :
2     def __init__(self, ring, num, den=None) :
3         if not den : den = ring.unit
4         if den == ring.zero : raise Exception("Fraction with denominator
5             of zero")
6         # Re-write the fraction as an irreducible
7         self.ring = ring
8         if num == ring.zero :
9             self.num = ring.zero
10            self.den = ring.unit
11        else :
12            gcd = euclidean_algorithm(ring, num, den)["gcd"]

```

```

12         self.num, _ = ring.euclideanDivision(num, gcd)
13         self.den, _ = ring.euclideanDivision(den, gcd)
14
15     def __add__ (self, other) :
16         return Frac(self.ring, self.num * other.den + other.num * self.den
17         , self.den * other.den)
18
19     def __sub__ (self, other) :
20         return Frac(self.ring, self.num * other.den - other.num * self.den
21         , self.den * other.den)
22
23     def __mul__ (self, other) :
24         return Frac(self.ring, self.num * other.num, self.den * other.den)
25
26     def __neg__ (self):
27         return Frac(self.ring, -self.num, self.den)
28
29     def __truediv__ (self, other) :
30         return Frac(self.ring, self.num * other.den, self.den * other.num)
31
32     def __str__ (self) :
33         if self.den == self.ring.unit : return str(self.num)
34         else: return f"\\frac{{{self.num}}}{{{self.den}}}"
35
36     def __eq__ (self, other) :
37         return self.num * other.den == self.den * other.num

```

Remark 5.15. At initialization, each fraction runs the euclidean algorithm on its numerator and denominator to write itself as an irreducible in the manner outlined in 5.10. This is a great re-use of the euclidean algorithm which is mainly intended for a user, but by not providing a writer can be used by the rings themselves.

Remark 5.16. In implementing the fraction ring and the integer ring, we have also implemented the ring of rational numbers, indeed $\mathbb{Q} \cong \text{Frac}(\mathbb{Z})$.

Example 5.17. We show the potential for computing complex results that programming rings over rings has through use of the extended euclidean algorithm on $E = \text{Frac}(\mathbb{Q}[X])[Y]$. We use the extended euclidean algorithm on $Y^2 + \frac{1}{-1X^3 + -1X}Y + \frac{1}{-1X^2 + -1}$ and $Y^2 + \frac{X^2 + -1X + 1}{-1X^3 + -1X}Y + \frac{1}{-1X^3 + -1X}$ using the console sequence in Appendix 6 to create output,

ExtendedEuclideanOutput.tex

Extended Euclidean Algorithm,

We use the Euclidean Algorithm to find the gcd of $Y^2 + \frac{1}{-1X^3 + -1X}Y + \frac{1}{-1X^2 + -1}$ and $Y^2 + \frac{X^2 + -1X + 1}{-1X^3 + -1X}Y + \frac{1}{-1X^3 + -1X}$

$$\begin{aligned}
 & Y^2 + \frac{X^2 + -1X + 1}{-1X^3 + -1X}Y + \frac{1}{-1X^3 + -1X} \\
 &= (1)(Y^2 + \frac{1}{-1X^3 + -1X}Y + \frac{1}{-1X^2 + -1}) + (\frac{\frac{1}{2}X + \frac{1}{2}}{\frac{1}{2}X^2 + \frac{1}{2}}Y + \frac{\frac{1}{2}X + \frac{-2}{2}}{\frac{1}{2}X^3 + \frac{1}{2}X}) \\
 & Y^2 + \frac{1}{-1X^3 + -1X}Y + \frac{1}{-1X^2 + -1} \\
 &= (\frac{\frac{1}{2}X^2 + \frac{1}{2}}{\frac{1}{2}X + \frac{1}{2}}Y + \frac{X}{-1X + 1})(\frac{\frac{1}{2}X + \frac{1}{2}}{\frac{1}{2}X^2 + \frac{1}{2}}Y + \frac{\frac{1}{2}X + \frac{-2}{2}}{\frac{1}{2}X^3 + \frac{1}{2}X}) + (0)
 \end{aligned}$$

Overall the gcd is $\frac{\frac{1}{2}X + \frac{1}{2}}{\frac{1}{2}X^2 + \frac{1}{2}}Y + \frac{\frac{1}{2}X + \frac{-2}{2}}{\frac{1}{2}X^3 + \frac{1}{2}X}$

We use the Extended Euclidean algorithm to find coefficients λ, μ s.t.

$$\lambda(Y^2 + \frac{X^2 + -1X + 1}{-1X^3 + -1X}Y + \frac{1}{-1X^3 + -1X}) + \mu(Y^2 + \frac{1}{-1X^3 + -1X}Y + \frac{1}{-1X^2 + -1}) = \frac{\frac{1}{2}X + \frac{1}{2}}{\frac{1}{2}X^2 + \frac{1}{2}}Y + \frac{\frac{1}{2}X + \frac{-2}{2}}{\frac{1}{2}X^3 + \frac{1}{2}X}$$

$$\begin{aligned}
 \frac{\frac{1}{2}X + \frac{1}{2}}{\frac{1}{2}X^2 + \frac{1}{2}}Y + \frac{\frac{1}{2}X + \frac{-2}{2}}{\frac{1}{2}X^3 + \frac{1}{2}X} &= (1)(Y^2 + \frac{X^2 + -1X + 1}{-1X^3 + -1X}Y + \frac{1}{-1X^3 + -1X}) \\
 &+ (-1)(Y^2 + \frac{1}{-1X^3 + -1X}Y + \frac{1}{-1X^2 + -1})
 \end{aligned}$$

Overall we have: $\lambda = (1)$ and $\mu = (-1)$

which has minor indentation changes to fit on the page.

Remark 5.18 (Faults). We can see two drawbacks to our implementation here, firstly, the process for inputting values is huge. Secondly, simplification in the fractions isn't as good as it could be. Indeed, in the output there is term $\frac{\frac{1}{2}X + \frac{1}{2}}{\frac{1}{2}X^2 + \frac{1}{2}}$ which could be simplified to $\frac{-X+1}{X^2+1}$ however, both are equally valid irreducible fractions and so our code is not equipped to decide which is a better representation.

6. IN CONCLUSION

We have explored Euclidean Domains, the Euclidean Algorithm and the Extended Euclidean Algorithm through an python Object Orientated Approach. While the code is not perfect, in particular in its input and it's output of fractions, it effectively uses the mathematical concepts outlined in this paper to conduct the algorithm over a variety of rings and create a reasonably well formatted output using latex, which starts failing when inputs get too long.

REFERENCES

- [1] David Barnes, *Lecture Notes*
- [2] Fraleigh, J. B., *A First Course in Abstract Algebra*, 7th ed., Addison-Wesley, 2003.
- [3] Gallian, J. A., *Contemporary Abstract Algebra*, 10th ed., Cengage, 2021.
- [4] Stark, H. M., *An Introduction to Number Theory*, MIT Press, 1978.
- [5] Lang, S., *Algebra*, Revised 3rd ed., Springer, 2002.

APPENDIX A. CONSOLE I/O LOGS FOR 5.17

```

1 "Please choose a Ring to operate in (Z (Integers), F[X] (Polynomials over
  a Field))"
2 F[X]
3 "Enter a letter to represent the indeterminate (typically X)"
4 Y
5 "Please choose a field to operate over (Frac[E] (Fractions over a
  Euclidean Domain), Z/p (Integers mod P))"
6 Frac
7 "Please choose a Ring to operate in (Z (Integers), F[X] (Polynomials over
  a Field))"
8 F[X]
9 "Enter a letter to represent the indeterminate (typically X)"
10 X
11 "Please choose a field to operate over (Frac[E] (Fractions over a
  Euclidean Domain), Z/p (Integers mod P))"
12 Frac
13 "Please choose a Ring to operate in (Z (Integers), F[X] (Polynomials over
  a Field))"
14 Z
15 "Enter the two elements to run the Extended Euclidean Algorithm on"
16 "Enter the first element:"
17 "Enter a degree for the polynomial"
18 2
19 "Enter the coefficient for Y^2 by entering a ring value:"
20 "Enter the numerator of the Fraction"
21 "Enter a degree for the polynomial"
22 0
23 "Enter the coefficient for X^0 by entering a ring value:"
24 1
25 "Enter the denominator of the Fraction"
26 "Enter a degree for the polynomial"
27 0
28 "Enter the coefficient for X^0 by entering a ring value:"
29 1
30 "Enter the coefficient for Y^1 by entering a ring value:"
31 "Enter the numerator of the Fraction"
32 "Enter a degree for the polynomial"
33 2
34 "Enter the coefficient for X^2 by entering a ring value:"
35 -1
36 "Enter the coefficient for X^1 by entering a ring value:"

```

```
37 1
38 "Enter the coefficient for X^0 by entering a ring value:"
39 -1
40 "Enter the denominator of the Fraction"
41 "Enter a degree for the polynomial"
42 3
43 "Enter the coefficient for X^3 by entering a ring value:"
44 1
45 "Enter the coefficient for X^2 by entering a ring value:"
46 0
47 "Enter the coefficient for X^1 by entering a ring value:"
48 1
49 "Enter the coefficient for X^0 by entering a ring value:"
50 0
51 "Enter the coefficient for Y^0 by entering a ring value:"
52 "Enter the numerator of the Fraction"
53 "Enter a degree for the polynomial"
54 0
55 "Enter the coefficient for X^0 by entering a ring value:"
56 -1
57 "Enter the denominator of the Fraction"
58 "Enter a degree for the polynomial"
59 3
60 "Enter the coefficient for X^3 by entering a ring value:"
61 1
62 "Enter the coefficient for X^2 by entering a ring value:"
63 0
64 "Enter the coefficient for X^1 by entering a ring value:"
65 1
66 "Enter the coefficient for X^0 by entering a ring value:"
67 0
68 "Enter the second element:"
69 "Enter a degree for the polynomial"
70 2
71 "Enter the coefficient for Y^2 by entering a ring value:"
72 "Enter the numerator of the Fraction"
73 "Enter a degree for the polynomial"
74 0
75 "Enter the coefficient for X^0 by entering a ring value:"
76 1
77 "Enter the denominator of the Fraction"
78 "Enter a degree for the polynomial"
79 0
80 "Enter the coefficient for X^0 by entering a ring value:"
81 1
82 "Enter the coefficient for Y^1 by entering a ring value:"
83 "Enter the numerator of the Fraction"
84 "Enter a degree for the polynomial"
85 0
86 "Enter the coefficient for X^0 by entering a ring value:"
```

```

87 -1
88 "Enter the denominator of the Fraction"
89 "Enter a degree for the polynomial"
90 3
91 "Enter the coefficient for X^3 by entering a ring value:"
92 1
93 "Enter the coefficient for X^2 by entering a ring value:"
94 0
95 "Enter the coefficient for X^1 by entering a ring value:"
96 1
97 "Enter the coefficient for X^0 by entering a ring value:"
98 0
99 "Enter the coefficient for Y^0 by entering a ring value:"
100 "Enter the numerator of the Fraction"
101 "Enter a degree for the polynomial"
102 0
103 "Enter the coefficient for X^0 by entering a ring value:"
104 -1
105 "Enter the denominator of the Fraction"
106 "Enter a degree for the polynomial"
107 2
108 "Enter the coefficient for X^2 by entering a ring value:"
109 1
110 "Enter the coefficient for X^1 by entering a ring value:"
111 0
112 "Enter the coefficient for X^0 by entering a ring value:"
113 1
114 "Extended Euclidean Algorithm Completed successfully, see
    ExtendedEuclideanOutput.tex"

```

APPENDIX A. FULL PYTHON CODE

rings.py

```

1 import operator
2 from extended_euclidean import euclidean_algorithm
3
4 # Helper function to get data from console
5 def tillValid(GetElement, repeatMessage) :
6     valid = False
7     while not valid :
8         valid = True
9         try :
10             e = GetElement()
11         except KeyboardInterrupt:
12             # We want keyboard interrupt to still exit the script
13             raise KeyboardInterrupt
14     except:
15         # If we have invalid input in GetElement
16         print(repeatMessage)
17         valid = False

```

```

18     return e
19
20 def getField() :
21     while True :
22         print("Please choose a field to operate over (Frac[E] (Fractions
23         over a Euclidean Domain), Z/p (Integers mod P))")
24         choice = input()
25         match choice.lower() :
26             case "z/p" | "zp" :
27                 return ZpRing()
28             case "q" | "frac" | "e" | "f" | "fr" | "fe" | "frac[e]" | "
29                 frac(e)" :
30                 return FractionRing()
31                 print("That is not a valid ring")
32
31 def getEuclideanDomain() :
32     while True :
33         print("Please choose a Ring to operate in (Z (Integers), F[X] (
34         Polynomials over a Field))")
35         choice = input()
36         match choice.lower() :
37             case "z" :
38                 return IntegerRing
39             case "f" | "f[x]" | "fx" | "f(x)":
40                 return PolyRing()
41                 print("That is not a valid ring")
42
43 class ZpRing :
44     def __init__(self, p=None) :
45         if not p :
46             print("Please choose p for Z/p")
47             p = tillValid(lambda: int(input()), "Please choose an integer
48             mod p")
49         self.p = p
50         self.unit = Zp(self.p, 1)
51         self.zero = Zp(self.p, 0)
52
53     def console_element (self) :
54         return Zp(self.p, tillValid(lambda: int(input()), f"Please choose
55         an integer mod {self.p}"))
56
57 class Zp:
58     def __init__(self, p, value) :
59         self.p = p
60         self.value = int(value) % p
61
62     def check_arith_match(self, other) :
63         if self.p != other.p : raise Exception("Modular arithmetic
64         mismatch")

```

```

62
63     def __add__ (self, other) :
64         self.check_arith_match(other)
65         return Zp(self.p, self.value + other.value)
66
67     def __sub__ (self, other) :
68         self.check_arith_match(other)
69         return Zp(self.p, self.value - other.value)
70
71     def __mul__ (self, other) :
72         self.check_arith_match(other)
73         return Zp(self.p, self.value * other.value)
74
75     def __neg__ (self):
76         return Zp(self.p, self.p - self.value)
77
78     def __str__ (self) :
79         return f"[{self.value}]_{{{self.p}}}"
80
81     def __truediv__(self, other) :
82         # Calculate the inverse by fermats little theorem
83         self.check_arith_match(other)
84         for _ in range(self.p - 2) :
85             self *= other
86         return self
87
88     def __eq__(self, other):
89         # Don't check arithmatch as we instead use n to check if values
90         # are equal
91         return self.value == other.value and self.p == other.p
92
93 class FractionRing :
94     def __init__(self, ring=None) :
95         if not ring :
96             ring = getEuclideanDomain()
97             self.unit = Frac(ring, ring.unit, ring.unit)
98             self.zero = Frac(ring, ring.zero, ring.unit)
99             self.ring = ring
100
101     def console_element (self) :
102         if self.ring == IntegerRing :
103             return tillValid(lambda: Frac(self.ring, *[int(x) for x in
104             input().split("/")]), "Enter a fraction in the form X/Y for integers X
105             and Y")
106
107         else :
108             print("Enter the numerator of the Fraction")
109             num = self.ring.console_element()
110             print("Enter the denominator of the Fraction")

```

```

109         den = self.ring.console_element()
110         return Frac(self.ring, num, den)
111
112 class Frac :
113     def __init__ (self, ring, num, den=None) :
114         if not den : den = ring.unit
115         if den == ring.zero : raise Exception("Fraction with denominator
of zero")
116         # Re-write the fraction as an irreducible
117         self.ring = ring
118         if num == ring.zero :
119             self.num = ring.zero
120             self.den = ring.unit
121         else :
122             gcd = euclidean_algorithm(ring, num, den)["gcd"]
123             self.num, _ = ring.euclideanDivision(num, gcd)
124             self.den, _ = ring.euclideanDivision(den, gcd)
125
126     def __add__ (self, other) :
127         return Frac(self.ring, self.num * other.den + other.num * self.den
, self.den * other.den)
128
129     def __sub__ (self, other) :
130         return Frac(self.ring, self.num * other.den - other.num * self.den
, self.den * other.den)
131
132     def __mul__ (self, other) :
133         return Frac(self.ring, self.num * other.num, self.den * other.den)
134
135     def __neg__ (self):
136         return Frac(self.ring, -self.num, self.den)
137
138     def __truediv__ (self, other) :
139         return Frac(self.ring, self.num * other.den, self.den * other.num)
140
141     def __str__ (self) :
142         if self.den == self.ring.unit : return str(self.num)
143         else: return f"\\frac{{{self.num}}}{{{{self.den}}}}"
144
145     def __eq__ (self, other) :
146         return self.num * other.den == self.den * other.num
147
148 # Ring of polynomials, Character is the letter used for the unknown and
ring is the underlying ring
149 class PolyRing :
150     def __init__ (self, ring=None, character=None) :
151         if not character :
152             print("Enter a letter to represent the indeterminate (
typically X)")
153         def getChar() :

```

```

154         c = input()
155         if len(c) != 1 or not c.isalpha() :
156             raise Exception("Not a character")
157         return c
158         self.indeterminate = tillValid(getChar, "Enter an alphabetical
character")
159
160         ring = getField()
161
162         self.ring = ring
163         self.zero = Polynomial(ring, [ring.zero], self.indeterminate)
164         self.unit = Polynomial(ring, [ring.unit], self.indeterminate)
165
166         def console_element (self) :
167             print("Enter a degree for the polynomial")
168             degree = tillValid(lambda: int(input()), "Enter an integer degree"
)
169
170             values = []
171             for i in range(degree,-1,-1) :
172                 print(f"Enter the coefficient for {self.indeterminate}^{i} by
entering a ring value:")
173                 values.insert(0, self.ring.console_element())
174             return Polynomial(self.ring, values, self.indeterminate)
175
176         # The euclidean division over polynomials is polynomial long division
177         def euclideanDivision(self, dividend, divisor) :
178             remainder = dividend
179             quotient = self.zero
180
181             while(divisor.length() <= remainder.length()) :
182                 # Create the monomial multiplier
183                 mult = Polynomial(self.ring, [self.ring.zero] * (remainder.
length() - divisor.length()) + [remainder.highestCoeff() / divisor.
highestCoeff()], self.indeterminate)
184                 remainder = remainder - (divisor * mult)
185                 quotient = quotient + mult
186
187             return (quotient, remainder)
188
189         # Not the traditional euclidean function in F[X] which is deg(f), but
is a valid one nonetheless
190         @staticmethod
191         def euclideanFunction(p) :
192             return p.length()
193
194     class Polynomial :
195         # We store polynomials as a + bx + cx^2 + ... + dx^n = [a b c ... d]
196         def __init__(self, ring, values, indeterminate="X") :
197             self.indeterminate = indeterminate

```

```

198     # The underlying ring
199     self.ring = ring
200     # Remove leading zeroes
201     while len(values) > 0 and values[-1] == ring.zero:
202         values.pop()
203     self.values = values
204
205     def length(self) :
206         return len(self.values)
207
208     def strTerm(self, coeff, pow) :
209         if coeff == self.ring.zero:
210             return ""
211         elif pow >= 1:
212             strCo = str(coeff) if coeff != self.ring.unit else ""
213             strX = f"{self.indeterminate}^{{{pow}}}" if pow > 1 else f"{
self.indeterminate}"
214             return strCo + strX
215         else :
216             return str(coeff)
217
218     def __str__(self) :
219         if self.values == [] :
220             # Return constant zero for zero polynomial
221             return str(self.ring.zero)
222
223         # Create string representation of polynomial
224         # TODO Check this is a smart implementation
225         rVals = [self.values[-i] for i in range(1, self.length() + 1)]
226         strP = [self.strTerm(rVals[i], len(rVals)-i-1) for i in range (0,
len(rVals))]
227         return " + ".join(filter(lambda s: s != "", strP))
228
229     # We use coeffwise to simplify the other methods
230     def coeffwise(self, other, op) :
231         length = max(self.length(), other.length())
232
233         def withLeadingZeroes(l, length) :
234             # Add leading zeroes till the list goes up to the desired
length
235             return l + [self.ring.zero] * (length - len(l))
236
237         p1, p2 = withLeadingZeroes(self.values, length), withLeadingZeroes
(other.values, length)
238         return Polynomial(self.ring, [op(p1[i], p2[i]) for i in range(
length)], self.indeterminate)
239
240     def __add__(self, other) :
241         return self.coeffwise(other, operator.add)
242

```

```

243     def __sub__(self, other) :
244         return self.coeffwise(other, operator.sub)
245
246     def __mul__(self, other) :
247         total = Polynomial(self.ring, [self.ring.zero], self.indeterminate
248     )
249         for i, x in enumerate(self.values) :
250             total = total + Polynomial(self.ring, [self.ring.zero] * i + [
251             x*y for y in other.values], self.indeterminate)
252         return total
253
254     def __eq__(self, other) :
255         # Efficient for polynomials of different lengths
256         if self.length() != other.length() :
257             return False
258         # Iterate over self and other simultaneously
259         for x, y in zip(self.values, other.values) :
260             if x != y :
261                 return False
262         return True
263
264     def __neg__(self) :
265         return Polynomial(self.ring, [-x for x in self.values], self.
266         indeterminate)
267
268     def highestCoeff(self) :
269         return self.values[-1] if self.length() > 0 else self.ring.zero
270
271 # IntegerRing is static as it doesn't need paramaters at initialisation
272 class IntegerRing :
273     zero = 0
274     unit = 1
275     @staticmethod
276     def console_element() :
277         return tillValid(lambda: int(input()), f"Please choose an integer"
278     )
279
280     @staticmethod
281     def euclideanFunction(x) :
282         return abs(x)
283
284     @staticmethod
285     def euclideanDivision(x,y) :
286         return (x // y, x % y)

```

extended_euclidean.py

```

1 def euclidean_algorithm(ring, e1, e2, writer=None) :
2     # Assign divisor and dividend where 0 keeps track of the originals
3     dividend0, divisor0 = (e1, e2) if ring.euclideanFunction(e1) >= ring.
    euclideanFunction(e2) else (e2, e1)

```

```

4     dividend, divisor = dividend0, divisor0
5
6     if writer :
7         writer.write(f"\\textbf{{We use the Euclidean Algorithm to find
the gcd of ${divisor0}$ and ${dividend0}$}} \\\\\\n")
8         writer.write("\\begin{align*} \\n")
9         results = []
10        remainder = None
11
12        while (remainder is None or remainder != ring.zero) :
13            quotient, remainder = ring.euclideanDivision(dividend, divisor)
14            results.append([dividend, quotient, divisor])
15
16            if writer :
17                writer.write(f"${dividend} &= ({quotient})({divisor}) + ({
remainder}) \\\\\\n")
18
19                dividend = divisor
20                divisor = remainder
21
22        gcd = dividend
23        if writer :
24            writer.write("\\end{align*} \\n")
25            writer.write(f"\\textbf{{Overall the gcd is ${gcd}$}} \\\\\\n")
26
27        return { "gcd":gcd, "results":results }
28
29 def extended_euclidean_algorithm(ring, e1, e2, writer=None) :
30     euclid = euclidean_algorithm(ring, e1, e2, writer)
31     gcd, results = euclid["gcd"], euclid["results"]
32     dividend0, divisor0 = (e1, e2) if ring.euclideanFunction(e1) >= ring.
euclideanFunction(e2) else (e2, e1)
33
34     # Remove result with zero remainder
35     results.pop()
36
37     # Design is discussed in the report
38     def extended_euclidean(n) :
39         dividend, quotient, divisor = results[n]
40         if n == len(results) - 1 :
41             lamb, mu = ring.unit, -quotient
42         else :
43             # Swap and calculate new mu
44             mu, lamb = extended_euclidean(n+1)
45             mu = mu - (quotient * lamb)
46
47         if writer :
48             writer.write(f"${gcd} &= ({lamb})({dividend}) + ({mu})({divisor
}) \\\\\\n")
49         return (lamb, mu)

```

```

50
51     if(len(results) != 0) :
52         if writer :
53             writer.write(f"\\textbf{{We use the Extended Euclidean
algorithm to find coefficients $\\lambda$, $\\mu$ s.t. $\\lambda ({
dividend0}) + \\mu ({divisor0}) = {gcd}$}} \\\\\\n")
54             writer.write("\\begin{align*} \\n")
55
56             lamb, mu = extended_euclidean(0)
57
58             if writer:
59                 writer.write("\\end{align*} \\n")
60                 writer.write(f"\\textbf{{Overall we have: $\\lambda = ({lamb})$
and $\\mu = ({mu})$}} \\\\\\n")
61         else :
62             # In trivial case, mu is the gcd
63             lamb, mu = ring.zero, ring.unit
64
65             if writer :
66                 writer.write(f"\\textbf{{We have trivially that $({lamb})({
dividend0}) + ({mu})({divisor0}) = {gcd}$}} \\\\\\n")

```

driver.py

```

1 from rings import getEuclideanDomain
2 from extended_euclidean import extended_euclidean_algorithm
3
4 # Setup ring-specific variables
5 ring = getEuclideanDomain()
6
7 print("Enter the two elements to run the Extended Euclidean Algorithm on")
8 print("Enter the first element:")
9 e1 = ring.console_element()
10
11 print("Enter the second element:")
12 e2 = ring.console_element()
13
14 path = "./ExtendedEuclideanOutput.tex"
15
16 # Open a file writer
17 writer = open(path, "w")
18 writer.write("\\textbf{{Extended Euclidean Algorithm,} \\\\\\n")
19 extended_euclidean_algorithm(ring, e1, e2, writer=writer)
20 print("Extended Euclidean Algorithm Completed successfully, see
ExtendedEuclideanOutput.tex")
21 writer.close()

```